2026년도 한국멀티미디어학회 추계학술발표대회 논문집 제29권 2호

프레임-스테이트 실행 모델 기반 비주얼 노벨 저작 프레임워크

유현수, 고정국
동명대학교 컴퓨터공학과
e-mail : lioie6478@gmail.com

A Visaul Novel Authoring Framework Based on a Frame-State Execution Model

Hyun-su Yu, Jeong-gook Koh
Dept. of Computer Engineering, Tongmyong University

요 약

본 논문은 비주얼 노벨 저작 시 연출과 로직의 결합 문제를 해결하는 엔진 독립 저작 프레임워크인 Visual Wave(VW)를 제안한다. VW는 시나리오·자원·시스템 계층 분리와 렌더 스테이트 테이블을 응용한 프레임-스테이트 실행 모델을 도입하여, 비종단 명령의 상태 누적과 종단 명령에서의 일괄 반영 및 순서 기반 상충 해소(Last-Write-Wins)를 수행한다. 상용 수준 비주얼 노벨 적용 결과, 변화분 선언 방식을 통해 대사 1자당 저장 용량을 기존 노드그래프 대비 1/6 수준(4.7 B)으로 줄이고 상태 중복률 0%를 달성하였다. 또한 화면당 약 0.16 ms의 뛰어난 런타임 성능을 입증하였으며, Godot 엔진에서의 수정 없는 구동을 통해 높은 엔진 독립성을 실증하였다.

1. 서론

비주얼 노벨(Visual Novel)은 대사를 중심으로 배경·캐릭터·음향·연출과 플레이어 선택에 따른 분기를 결합한 장르로, 한 편은 수천 줄의 대사와 수많은 연출 지시, 다국어 텍스트, 루트 간 이동으로 이루어진다[1]. 전통적 저작 방식은 연출 로직을 게임 엔진 코드에 직접 작성하거나 작품마다 임시 스크립트로 기술하는 것인데, 이는 서사 콘텐츠를 특정 엔진 및 구현에 강하게 결합시켜 작가가 프로그래밍 구조를 깊이 파악해야 하고 연출 코드의 재사용을 어렵게 만든다.

본 연구는 동일한 작품을 이전에 노드그래프 기반 도구로 저작한 경험과 문제의식에서 출발하였다. 해당 도구는 한 화면(프레임)을 전체 상태 객체로 저장하여 상태가 변경되지 않아도 프레임마다 11개 상태 필드를 반복 기록하였고, 리소스를 Unity 에디터 직렬화 참조로 연결해야 했으며, 기계 생성 형식이라 전용 도구 내부에서만 편집할 수 있었다[1, 2].

제안하는 프레임워크 Visual Wave는 대본과 리소스를 모두 텍스트 전용 언어(DSL)로 표현하여, ① 변경되는 상태의 변화분만 선언함으로써 데이터 용량을 획기적으로 절감하고, ② 리소스를 텍스트 경로로 기술하여 런타임에 해석함으로써 재바인딩 시 에디터 재연결이나 코드 재빌드가 필요 없으며, ③ 평문 기반이므로 전용 VS Code 확장을 통해 구문 강조·자동완성·인레이 힌트를 제공하는 환경을 구축하였다.

이러한 구조적 접근의 핵심 착안점은 실시간 그래픽스 파이프라인의 렌더 스테이트 테이블(Render State Table) 메커니즘이다. 그래픽스 개발자가 블렌딩, 깊이 등 화면 상태를 선언적으로 설정한 후 드로우 콜(Draw Call)을 발행하여 하나의 프레임을 최종 연산하듯이[4], 비주얼 노벨의 한 화면 역시 여러 채널의 상태가 겹쳐진 결합 결과로 간주하여 상태 누적 후 대사 및 선택지 시점에서 일괄 확정하도록 설계하였다.

본 논문의 핵심 기여는 다음과 같다. 첫째, 작성-편집-구동을 포괄하고 콘텐츠를 시나리오·자원·시스템의 3개 계층으로 분리하여 작가와 프로그래머의 관심사를 명확히 구분하는 완결된 저작 프레임워크를 제안한다. 둘째, 비종단 명령의 상태 누적, 종단 명령에서의 결정론적 일괄 반영, 상충 명령의 순서 기반 최종 반영(Last-Write-Wins)으로 정의되는 프레임-스테이트 실행 모델을 제시한다.

2. 연구 방법

2.1 Visaul Wave 프레임워크 구성 및 계층 분리

Visual Wave는 특정 엔진에 의존하지 않고, 그림 1과 같이 DSL 해석과 프레임-스테이트 실행을 담당하는 코어 라이브러리, 화면 표현을 담당하는 엔진별 런타임 라이브러리(Unity, Godot), 그리고 VS Code LSP 기반 저작 지원 확장의 세가지 컴포넌트로 구성된다.

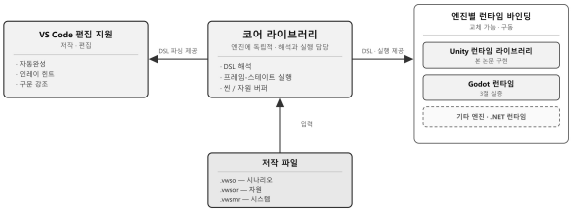


그림 1. Visual Wave 프레임워크

저작자는 콘텐츠를 다음 세 가지 평문 파일로 나누어 작성한다.

(1) 시나리오 스크립트(.vwso) : 장면 분기 및 연출 명령 기술
(2) 자원 정의 파일(.vwsor)는 숫자 ID와 실제 리소스 상대 경로·별칭 및 캐릭터 배치 프리셋 대응
(3) 시스템 정의 파일(.vwsmr) : 전역 변수와 지원 언어 선언

2.2 시나리오 DSL 문법 사양

시나리오 DSL은 한 줄을 하나의 명령으로 인식하며 줄 앞머리의 접두사로 종류를 판별한다[2]. 각 명령에는 ?로 시작하는 연출 옵션과 !로 시작하는 매크로를 조합할 수 있으며, 주요 사양은 표 1과 같다.

표 1. Visual Wave 시나리오 명령 분류 및 사양

분류	대표 문법	의미
장면	SS#Route_NN / ES	루트 시작/장면 종료
배경	BG_NN ?EF_Fade	배경 자원 지정 효과 전환
캐릭터	CH_NN_EE ?RP_Speaker / DCH_NN	캐릭터 추가·주목/삭제
대사	Line_NN ?EF_.. !LA_KO "…" !LA_EN "…"	화자의 다국어 대사 출력
대사 배경	LineBG_NN ?EF_Fade	대사창 배경 지정
선택지	BCH ?CD_[…] !LA_.. "…" !GV_… ?JMP_Route_NN	조건·대사·변수·점프 분기
장면 점프	JMP_Route_NN	다른 장면·루트로 이동
음향	BGM_NN / BGM_OFF / SFX_NN	음악 지속/중단/효과음
연출	PPEF_.., CG_NN, CameraEF_.., PostProcess_NN	프롭·CG·카메라·후처리
시스템	!GV_Name_Set "값"	전역 변수 조작

2.3 프레임-스테이트 실행 모델 및 비동기 연출 동기화

프레임-스테이트 실행 모델에서 실행기는 그림 2와 같이 시나리오 스크립트를 순차적으로 해석하며 배경·캐릭터·음향·연출 등 화면을 즉시 확정하지 않는 비종단 명령을 프레임 상태에 누적한다[2]. 이후 대사(Line), 선택지(BCH), 장면 끝(ES) 등 종단 명령에 도달하면 누적된 프레임 상태를 채널별 핸들러로 일괄 반영한다. 동일 채널의 상충 명령은 순서 기반 최종 반영(Last-Write-Wins) 규칙으로 해소된다.

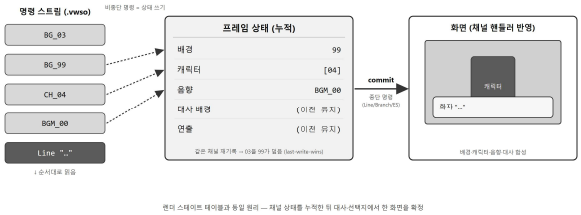


그림 2. 프레임-스테이트 실행 모델

종단 명령 발행 시 채널별 핸들러는 배경→캐릭터→음향→연출→대사의 결정론적 순서로 상태 반영 지시가 디스패치된다. 카메라 쉐이크나 페이드 같은 시간 경과형 비동기 연출은 목표 상태 확정과 분리되어, 목표 상태 설정 후 핸들러 내부의 코루틴(Coroutine)과 트윈(Tween) 엔진을 통해 지정된 소요 시간에 따라 전개된다.

2.4 LSP 기반 저작 편집 지원

숫자 식별자(CH_04_01, BG_01) 사용에 따른 가독성 저하를 방지하기 위해 VS Code 전용 LSP(Language Server Protocol) 서버를 제공한다[3]. 서버는 시나리오·자원·시스템 파일을 실시간으로 인식하여 자원 식별자 자동 완성, 마우스 오버 시 자원 정보 표시, 숫자 식별자 옆에 별칭을 덧대는 인레이 힌트, 구문 강조를 수행한다. 그리고 LSP 미지원 에디터나 코드 리뷰 환경을 고려하여 파서 레벨에서 문자열 별칭 식별자의 혼용 표기를 허용하도록 확장성을 부여하였다[2].

3. 연구 결과

3.1 상용 비주얼 노벨 적용 및 저작 생산성 검증

제안 프레임워크를 상용 수준의 비주얼 노벨(4개 루트, 55개 장면, 3,035행 명령, 2,497행 대사)에 적용하였다[2]. 단일 자원 정의 파일 하나로 배경 9종, 캐릭터 12종(이미지 바인딩 45개), BGM 10종, SFX 13종, CG 18종을 모아 통합 관리하였다. 세부 적용 규모는 표 2와 같다.

표 2. 샘플 작품 적용 규모 측정 지표

항목 구분	적용 수량 및 세부 값
루트 수/장면 수/파일 수	4개 루트/55개 장면/55개 파일
전체 명령행(대사 명령)	3,035행 (2,497행)
배경/캐릭터 추가/삭제	38회/79회/39회
배경음악/효과음/CG 연출	75회/44회/13회
선택지/장면 점프	28회/26회
자원: 배경/캐릭터(바인딩)	9종/12종 (45개 이미지)
자원: BGM/SFX/CG/프리셋	10종/13종/18종/13개

전역 변수/언어	6개/1개(KO, 다국어 인라인 지원)

실제 적용 결과, 2,497행의 대사가 12종의 캐릭터 식별자만 참조하므로 시나리오 파일의 수정을 배제한 채 .vwsor 파일의 리소스 경로만 수정하여 45개 캐릭터 표정 이미지를 에디터 재연결이나 코드 재빌드없이 즉시 재바인딩하는 높은 저작 효율성을 확인하였다.

3.2 런타임 성능 측정

.NET Framework 4.8 Release 빌드 환경에서 55개 시나리오 파일의 적재와 파싱 시간 측정 결과, 전체 파싱 소요 시간은 약 11.5 ms로 측정되었다. 화면 1회당 프레임 상태 구성 비용은 평균 약 0.16 ms로 측정되어 60 fps 처리 예산(약 16.7 ms)을 크게 밑돌았다. 관리 힙(Managed Heap) 메모리 점유량도 약 0.48 MB에 불과하여 실시간 실행 오버헤드가 무시할 수 있는 수준임을 입증하였다.

3.3 기존 저작 도구와의 정량·구조적 비교

동일 작품을 저작하였던 노드그래프 기반 도구[2] 및 대표적 텍스트 DSL 도구들과 정량적·구조적 지표 비교 결과는 표 3 및 표 4와 같다.

표 3. 동일 작품 대상 노드그래프와의 정량 비교

지표 항목	노드그래프 도구	제안 방식(VW)
대사 1자당 저장 용량	약 30.0 B	약 4.7 B(1/6 수준)
상태 중복률	85.4 %	0 % (변화분만 기재)
화면 1회 기재량	고정 스키마 11개	평균 1.14개

표 4. 주요 비주얼 노벨 저작 도구와의 구조 비교

도구	독립 코어	연출 명령	평문 DSL	자원 분리	다국어
VW	예	예	예(LSP)	예(ID)	인라인
Ren'Py[5]	아니오	예	예	인라인	별도파일
Naninovel [6]	아니오	예	예	인라인	별도파일
Ink[7]	예	태그위주	예	예	별도파일
Fungus[8]	아니오	예(GUI)	아니오	인라인	별도파일

3.4 Godot 4.7 엔진 독립성 실증

코어 라이브러리의 엔진 독립성을 실증하기 위해 Godot 4.7 Vulkan 렌더러 기반 환경에서 동일 코어 라이브러리와 시나리오·자원·시스템 파일을 수정 없이 구동하는 최소 사례를 그림 3과 같이 구현하였다. 코어 라이브러리는 .NET 런타임 환경에서 Godot C#과 연동되었으며, 화면 표현 계층만 Godot 노드로 연결하여 정상 작동함을 보였다.



그림 3. 엔진 독립성 실증(Godot 엔진 버전)

4. 결론

본 논문에서는 비주얼 노벨 시나리오의 작성–편집–구동을 포괄하는 저작 프레임워크와 프레임-스테이트 실행 모델을 제안하였다. 콘텐츠를 시나리오·자원·시스템의 3계층으로 분리하여 작가와 프로그래머의 관심사를 분리하고, 렌더 스테이트 테이블 원리의 상태 누적·일괄 반영·최종 반영 메카니즘을 구축하였다.

상용 수준 비주얼 노벨 적용을 통해 대사 1자당 저장 용량을 기존 노드그래프 대비 1/6인 4.7 B 수준으로 압축하고, 화면당 구성 비용 약 0.16 ms의 경량성을 입증하였고, Unity와 Godot 환경에서의 교차 구동으로 엔진 독립성을 보였다.

향후 과제는 저작 적합성 사전 검사 컴파일러 도입 및 사용자 평가(User Study)를 추진할 예정이다.

참고문헌

[1] ALCOHOL FREE, 비주얼 노벨 게임.
[2] 유현수, “AlcoholFree-PlotTree,” GitHub, 2026. https://github.com/HyunsuYu/AlcoholFree-PlotTree
[3] Microsoft, "Language Server Protocol Specification," https://microsoft.github.io/language-server-protocol/
[4] T. Akenine-Möller, E. Haines, N. Hoffman, et al., Real-Time Rendering, 4th ed. CRC Press, 2018.
[5] T. Rothamel, "Ren'Py Visual Novel Engine," https://www.renpy.org/
[6] Naninovel, "Naninovel—Visual Novel Engine for Unity," https://naninovel.com/
[7] inkle, "ink—inkle's narrative scripting language," https://www.inklestudios.com/ink/
[8] snozbot, "Fungus—Visual storytelling and scripting tool for Unity," https://github.com/snozbot/fungus/
[9] M. Fowler, Domain-Specific Languages, Addison-Wesley, 2010.